

Оценка затрат на разработку программного обеспечения с использованием нечеткой логики Базовая система в контексте масштабируемой гибкой методологии

Hung Q. Tran CAE
USA, FL, США

Аннотация: В контексте Scaled Agile Framework (SAFe) оценка трудозатрат на разработку программного обеспечения представляет собой критически важную задачу. Правильная оценка трудозатрат позволяет команде разработчиков планировать задачи разработки и прогнозировать время и стоимость, необходимые для выпуска программных продуктов. Кроме того, это позволяет команде разработчиков определить достаточную численность персонала (т.е. возможности команды) на этапе планирования итерации программы (PI).

Справка: SAFe — это фреймворк для масштабирования Agile-методологии в масштабах всего предприятия. На уровне команды разработчиков он предоставляет рекомендации по определению, созданию, тестированию и развертыванию программных решений в сжатые сроки. Agile-команды отвечают за предоставление результатов разработки программного обеспечения, соответствующих потребностям и ожиданиям клиентов. Часть этой ответственности заключается в оценке трудозатрат на выполнение работы.

Команды, работающие по гибкой методологии, разбивают требуемую работу на истории, представляющие собой краткие описания небольших фрагментов (функций) желаемой функциональности программного обеспечения. Трудозатраты на разработку и поставку истории оцениваются с помощью очков истории, которые представляют собой числовое значение, отражающее сочетание сложности работы, объема работы и неопределенности/риска реализации. Концепция очков истории проста, но часто сложна в применении. Основная сложность заключается в том, что очко истории представляет собой абстрактную субъективную величину. Многие разработчики программного обеспечения считают сложным соотносить очки истории со степенью сложности, объема и неопределенности трудозатрат.

Цель: В этой статье мы рассмотрим процесс разработки программного обеспечения в контексте SAFe, в частности, как Agile-команды обычно оценивают трудозатраты, используя Story Points. Затем мы предложим новый подход к оценке трудозатрат. Этот новый метод в значительной степени использует систему на основе нечеткой логики.

метод искусственного интеллекта, который имитирует процесс человеческого рассуждения на этапе оценки трудозатрат гибкими командами разработчиков программного обеспечения.

Заключение: Модель оценки трудозатрат на разработку программного обеспечения, включающая нечеткую логику, может помочь группам разработчиков программного обеспечения преодолеть проблему совместной оценки нескольких атрибутов.

Ключевое слово: масштабируемая гибкая методология; оценка усилий; точки повествования; нечеткая логика.

Дата подачи: 15-01-2020

Дата принятия: 03.02.2020

I. Введение Оценка

разработки программного обеспечения представляет собой процесс определения необходимых усилий для разработки, внедрения и поддержки программного обеспечения на основе требований заказчика. Синхал и Верма [1] опубликовали всесторонний обзор различных методов оценки программного обеспечения. В своей статье авторы изложили множество общепринятых методов, которые доступны в настоящее время. Как правило, методы оценки программного обеспечения можно разделить на следующие категории: алгоритмические, неалгоритмические и гибридные. Гибридная модель — это просто комбинация алгоритмических и неалгоритмических методов оценки [2]. Наиболее популярной моделью оценки алгоритма является конструктивная модель стоимости Бозма (COCOMO) [3]. Эта модель алгоритма требует входных данных модели, таких как оценка количества строк кода (LOC) или сложности реализации программного обеспечения. Такие атрибуты часто трудно оценить на ранних стадиях процесса разработки. Из-за этого ограничения многие исследователи изучали неалгоритмические методы оценки, которые основаны на мягких вычислениях, включая метод нечеткой логики [4][5][6].

Дин Леффингвелл впервые представил Scaled Agile Framework (SAFe) в 2011 году. SAFe — это проверенный в отрасли метод, который был разработан для помощи в решении некоторых проблем, с которыми сталкиваются крупные организации при применении методологии Agile.

Цель данной статьи — представить новый подход — систему на основе нечеткой логики, которую команды Agile-разработчиков могут использовать для оценки трудозатрат в контексте SAFe. Сначала мы рассмотрим метод нечеткой логики и его основные правила. Затем мы объясним процесс разработки и то, как этот метод можно применять для оценки трудозатрат на разработку программного обеспечения.

II. Предыстория

2.1. Масштабируемая гибкая структура

Структура SAFe имеет три основных уровня, которые централизуют стратегические темы организации: портфель, программа и команда [7].

- Портфель SAFe включает в себя людей и процессы, необходимые для создания решения, которое позволяет организации для достижения ее стратегических целей.
- Программа SAFe включает в себя команды разработчиков и другие ресурсы, предназначенные для текущего развития системы. Миссия развития. Её особая роль — помогать людям следовать общей миссии.
- Команда SAFe включает в себя роли, виды деятельности и процессы, которые команды Agile-разработки используют для создания и поставку рабочие продукты.

Agile-команды являются основой SAFe, поскольку они выполняют большую часть работы по внедрению. Фактически, все действия по разработке программного обеспечения происходят на уровне команды. Agile-команды состоят из кросс-функциональной группы разработчиков, отвечающих за определение, создание, тестирование и поставку рабочих продуктов в течение коротких итерационных периодов (т. е. Agile-спринтов). Их основная функция — предоставлять результаты, соответствующие потребностям и ожиданиям заказчика. Для этого они выполняют следующие действия [8]: - Декомпозиция «функций» рабочего продукта для определения «историй» (истории — это краткие описания функций).

(сохраняются в рамках отставания команды.)

- Оцените трудозатраты.
- Определить соответствующую техническую конструкцию, отвечающую потребностям и ожиданиям заказчика.
- Взять на себя обязательство выполнить работу, которую можно выполнить в рамках итерации (например, Agile-спринта). Продолжительность итерационного спринта обычно составляет 2 недели.
- Выполнить необходимые работы.
- Проверьте ожидаемую функциональность.
- Развертывание рабочих продуктов.

Эти задачи крайне важны для Agile-команды. Однако самой сложной и в то же время критически важной для Agile-команд является оценка трудозатрат. Правильная оценка трудозатрат позволяет команде успешно планировать и брать на себя обязательства по объёму работы, который, по её мнению, она может выполнить в течение цикла итерации программы.

2.2. История пользователя и точки истории.

В контексте управления проектами Agile бэклог команды представляет собой все потенциальные рабочие элементы команды. Эти рабочие элементы могут включать в себя истории пользователя, технические элементы и нефункциональные требования. Элементы, представленные в бэклоге команды, должны быть оценены с учетом уровня усилий, которые, по мнению команды, необходимы для их выполнения в соответствии с определением «готовности» и критериями приемки. Оценка трудозатрат имеет решающее значение, поскольку она дает команде руководство по планированию своей рабочей нагрузки в соответствии с приоритетами и бизнес-ценностями предприятия. Для успешного планирования работы Agile-команда должна учитывать два важных фактора: 1. Способность команды выполнить работу для следующего инкремента программы. 2. Скорость команды,

которая показывает, насколько быстро команда может выполнить элементы в бэклоге.

Баллы за истории используются в управлении и разработке проектов Agile для оценки уровня усилий [9]. Они представляют собой метрику, используемую для выражения уровня усилий, необходимых для реализации и поставки историй из бэклога команды [10]. По определению, баллы историй не коррелируют напрямую с фактическими часами; фактически, не существует линейной зависимости между баллами историй и часами, необходимыми для выполнения работы. Таким образом, это позволяет Agile-команде абстрактно мыслить об усилиях, необходимых для выполнения работы. Когда Agile-команда использует баллы историй для оценки усилий на разработку, оценка должна учитывать все три основных фактора совместно: сложность истории, размер работы, а также неопределенность и риск. Было замечено, что сложность оценки трудозатрат с помощью баллов историй значительно возрастает, когда команде необходимо учитывать все три основных фактора совместно.

В SAFe оценка трудозатрат обычно выполняется относительно общей начальной базовой линии. Поэтому команда должна определить базовую историю, относительно которой будут оцениваться все последующие истории. Базовая история обеспечивает одинаковую отправную точку для всех членов команды и обозначается как одна точка истории [7]. Базовая история описывает базовую функциональность, которая может быть такой простой, как «трудозатраты на написание командного файла или создание тестового случая».

Наконец, в оценке трудозатрат должны участвовать все члены команды разработки, поскольку каждый член команды имеет свой взгляд на работу, необходимую для выполнения и поставки истории. Изначально команда устанавливает последовательность чисел, которая будет использоваться для оценки трудозатрат. Модифицированный формат Фибоначчи очень популярен и обычно используется командами, работающими по Agile: 0,5, 1, 2, 3, 5, 8, 13, 20... Метод, обычно используемый для оценки трудозатрат в баллах истории, называется «Планирование покера» и основан на консенсусе всех членов команды и использует карты покера с присвоенными числами, представляющими ценность (баллы истории). Чтобы начать планирование

На сессии команда рассматривает историю из бэклога и кратко её обсуждает. После обсуждения участники команды выбирают карту со значением, соответствующим их оценке, после чего все карты открываются. Номера карт, показанные на экране, представляют собой количество баллов, которые, по мнению каждого участника команды, должны быть выделены для реализации и доставки этой истории, и которые были оценены относительно базовой пользовательской истории. Если оценки всех участников совпадают, команда принимает эту оценку и переходит к следующему пункту в бэклоге. Если нет, участники команды делятся своим обоснованием и обсуждают его, а затем повторяют процесс оценки в покере планирования, пока не достигнут консенсуса.

2.3. Нечёткая логика.

Нечёткая логика представляет собой метод искусственного интеллекта (ИИ), который может быть использован для имитации человеческого мышления. В отличие от обычной булевой логики «истина» или «ложь» (1 или 0), нечёткая логика представляет собой подход к вычислениям, основанный на «степени истинности». Этот тип логики оперирует значениями, которые выходят за рамки простого «истина» или «ложь». Более того, при использовании лингвистических переменных они могут быть представлены степенью истинности. или ложь. По этой причине нечеткая логика может обрабатывать сложные задачи управления с низкими вычислительными затратами и при этом сохранять тонкий аспект человеческого мышления. Фактически, управление на основе нечеткой логики — это не просто еще один метод управления — оно представляет собой генератор качественного поведения. По сравнению с другими интеллектуальными методами, системы на основе нечетких правил просты в проектировании и реализации, поскольку требуют только использования набора правил, извлеченного из базы знаний [11]. Экспертная система на основе нечетких правил содержит нечеткие правила, определенные ее базой знаний. Она выводит выводы или решения из входных данных пользователя и процесса нечеткого рассуждения. Экспертная система на основе правил использует очень описательные лингвистические переменные термины (например, «низкий», «умеренный», «высокий») для работы с входными и выходными данными, как это сделал бы человек-оператор. Более того, база знаний может расти постепенно, поэтому производительность системы улучшается по мере ее разработки.

Процесс разработки классической экспертной системы показан на рисунке 1. Инженеры устанавливают диалог или проводят собеседование с экспертом в предметной области (ЭСМ). Цель — собрать знания эксперта. Затем эти знания кодируются для создания базы данных на основе правил. ЭСМ проверяет и оценивает базу данных на основе правил, чтобы внести необходимые корректировки. Этот процесс проходит множество итераций, пока база данных знаний не будет полностью сформирована.

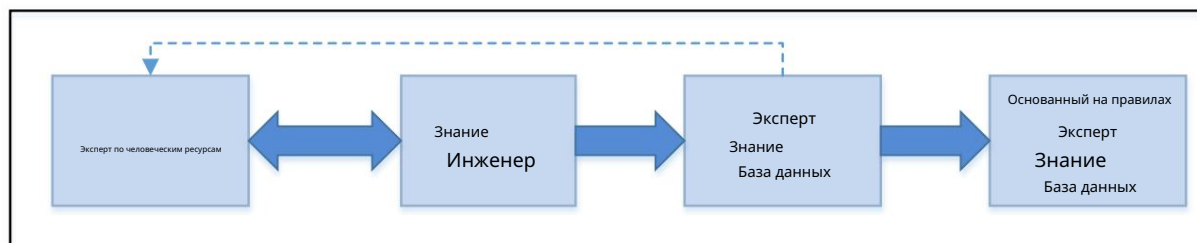


Рисунок 1. Разработка экспертной системы

2.4. Функции принадлежности

Функции принадлежности являются центральным компонентом систем на основе нечеткой логики. Функцию принадлежности можно определить как функцию, которая указывает степень принадлежности заданного входного значения множеству. Нечеткая логика была сформулирована Лофти-Заде в 1965 году [12] как парадигма вычислений, основанная на том, как люди думают. В математике нечеткая логика представляет собой способ обработки элементов, которые частично находятся во множестве. Как правило, множество представляет собой коллекцию элементов, которые имеют общие характеристики, например, множество «объема работы». Объем работы можно определить как «высокий», когда число строк, которые нужно произвести, превышает 10 000. Это множество можно графически представить, как на рисунке 2. Основной характеристикой четкого множества является то, что элемент либо является членом множества, либо нет. Не имеет значения, сколько строк кода — 10 000 или 20 000 — обе ситуации представляют собой «высокий» уровень усилий. С другой стороны, 9990 — это «низкий», хотя и всего на 10 строк кода меньше предельной длины. Эта математическая функция хорошо работает для двоичных операций, но не в реальных ситуациях.

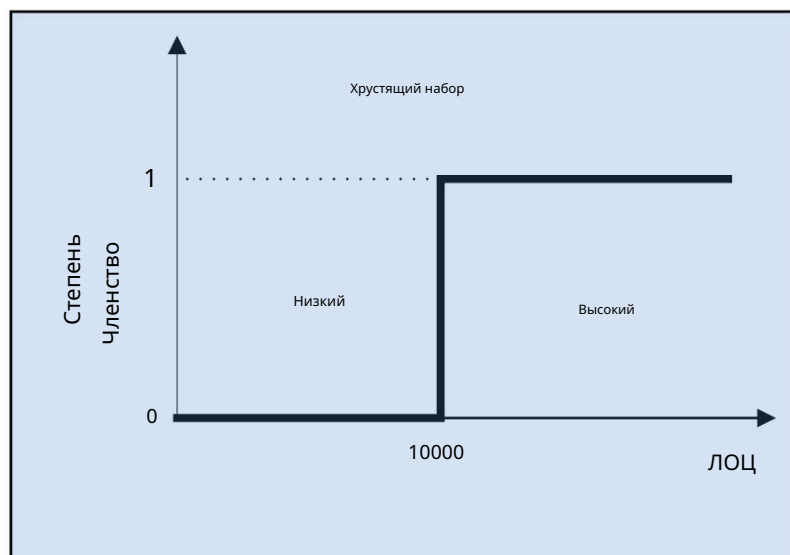


Рисунок 2. Объем работы — четкий набор

В отличие от классического множества, нечёткое множество допускает частичную принадлежность элементов к множеству. Эта концепция графически представлена на рисунке 3, где показано отсутствие чёткого разделения между категориями «низкий», «средний» и «высокий». Переход между этими категориями плавный. Чтобы использовать компьютер для обработки лингвистических переменных, таких как «низкий» или «высокий», чёткие входные данные можно преобразовать в нечёткие выходные значения, используя простую функцию принадлежности, аналогичную показанной на этом рисунке.

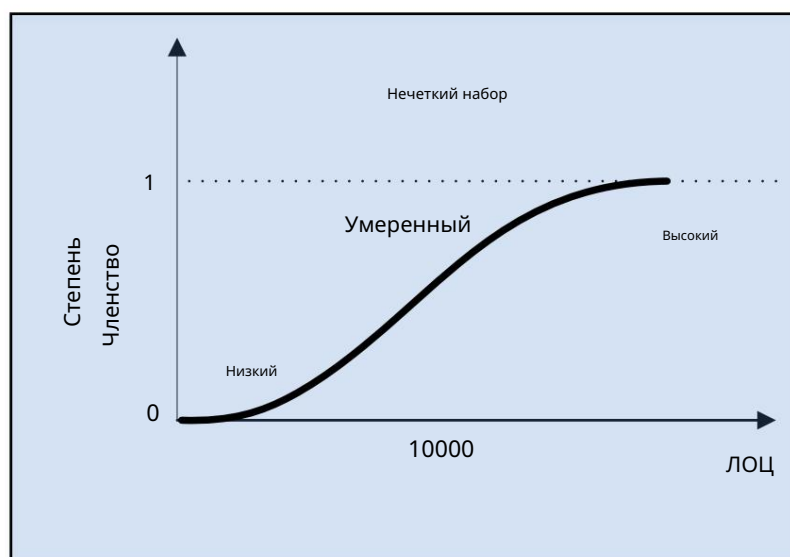


Рисунок 3. Объем работы — нечеткое множество

Подводя итог, можно сказать, что нечёткая логика может использоваться для работы с концепцией частичной истины, в отличие от традиционной булевой логики, где событие должно быть либо истинным, либо ложным. Как следует из названия, нечёткая логика представляет собой логику, лежащую в основе методов рассуждения, которые являются приблизительными, а не точными. Она возникла из того факта, что человеческие рассуждения, особенно рассуждения, основанные на здравом смысле, по своей природе приблизительны.

III. Система нечеткого вывода

Система нечёткого вывода — это процесс преобразования нескольких чётких входных данных в систему для получения чётких выходных данных с использованием нечётких множеств, основанных на правилах. Базовая структура нечёткой системы вывода состоит из трёх основных концептуальных

- элементов: 1. Множество, основанное на правилах, которое содержит набор нечётких правил; 2. Определение функций принадлежности, используемых в нечётких правилах.

3. Механизм рассуждения, который выполняет процедуру вывода для получения разумного заключения и решение

Концепция нечёткого логического управления (НЛУ) может быть представлена в виде нечёткого логического управления (НЛУ), которое было выведено из теории управления и основано на математических моделях процесса управления с разомкнутым контуром. НЛУ успешно применяется в различных практических приложениях, как простых, например, управление температурой в помещении и циклами стиральных машин, так и сложных, например, энергетические системы или ядерные реакторы. Компоненты НЛУ показаны на рисунке 4.

Три компонента FLC: 1. Фазификация чётких

входных параметров, представляющая собой процесс преобразования чётких входных параметров в подходящие лингвистические значения, представленные набором правил нечёткой логики. Нечёткие правила, содержащиеся в наборе правил нечёткой логики, были сформированы на основе базы знаний. База данных содержит определения, используемые для определения лингвистических правил управления и данных. Эти правила характеризуют логику управления экспертов предметной области.

2. Механизм нечеткого вывода представляет собой процессор FLC и имитирует человеческое принятие решений на основе создана база нечетких правил.

3. Дефазификация — это процесс обратного преобразования нечетких лингвистических параметров в требуемые четкие параметры. для процесса управления.

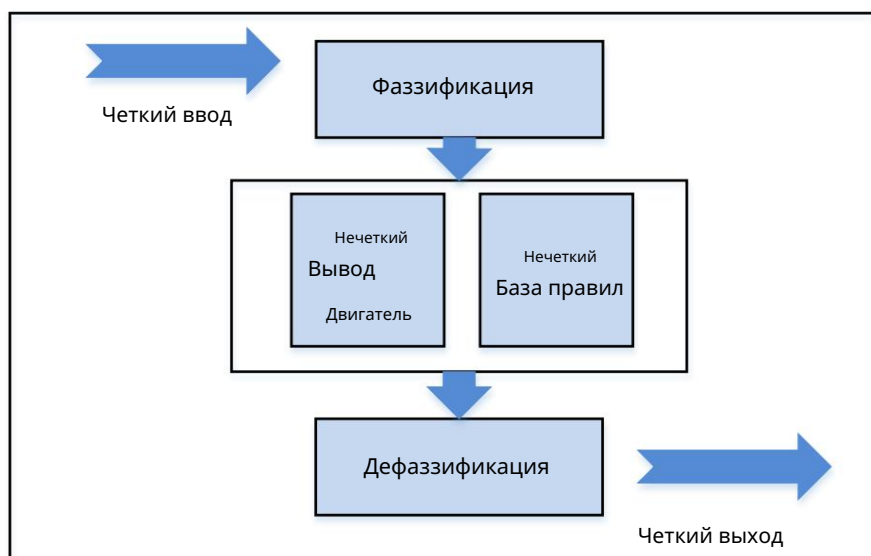


Рисунок 4. Контроллер нечеткой логики

Существует три основных типа систем нечёткого вывода (FLC): Мамдани [13], Сугено [14] и Цукamoto [15]. В данной статье мы будем использовать систему нечёткого вывода Мамдани для реализации правил ИИ для оценки эффективности программного обеспечения.

Метод Мамдани расширяет нечеткий контроллер для использования интервалов и лингвистических значений в качестве входных данных. Например, вместо выражения входных данных типа «Время записи файла пакетной команды составляет 1 час», эквивалентное правило нечёткой логики будет выглядеть так: «Время записи файла пакетной команды мало». Последнее выражение лучше соответствует человеческому мышлению. Система Мамдани FLC представляет собой нечёткий контроллер с N входами $X_1, X_2, \dots, X_n$ и одним выходом Y , с правилом нечёткой логики типа «Если X_1 равно A_1 или X_2 равно B_2 , то Y равно C_1 ».

Чтобы проиллюстрировать реализацию алгоритма Мамдани FLC, мы рассмотрели типичную задачу с двумя входными данными (X = сложность, Y = неопределённость), одним выходным данными (Z = трудозатраты на разработку) и набором из трёх правил. В этом примере мы использовали функции принадлежности трапециевидной или треугольной формы. В различных приложениях используются многие другие формы, например, гауссовая или колоколообразная [16]. Обычно конкретная форма выбирается в зависимости от области применения и доступности данных.

Для данного примера рассматривался следующий набор правил:

- Если сложность и неопределенность высоки, то и усилия по разработке будут высоки.

- Если сложность или неопределенность умеренные, то и усилия по разработке умеренные.

- Если сложность высока, то и трудозатраты на разработку высоки.

«X» — это нечеткое множество «сложности», «Y» — это нечеткое множество «неопределенности», а «Z» — это нечеткое множество «усилий по разработке».

Кроме того, по шкале от 0 до 10, предполагая, что сложность X равна 4, а неопределенность Y равна 8, необходимо вычислить выход Z (например, трудозатраты на разработку).

Шаг 1: Фазификация

Фазификация заключается в проецировании чёткого входного значения (X) на соответствующие нечёткие множества, как показано на рисунке 5. Проекция чёткого входного значения « X » на нечёткую переменную « A » математически выражается формулой $\mu(X = A)$. Таким образом, входная сложность $X = 4$ проецируется на нечёткие переменные «низкая» со значением 0,5, «умеренная» со значением 0,4 и «высокая» со значением 0,0. Значения 0,5, 0,4 и 0,0 представляют степень принадлежности X нечётким переменным «низкая», «умеренная» и «высокая» соответственно. Аналогично значения 0,1 и 0,7 представляют степень принадлежности неопределенности $Y = 7$ нечётким переменным «низкая» и «высокая» соответственно.

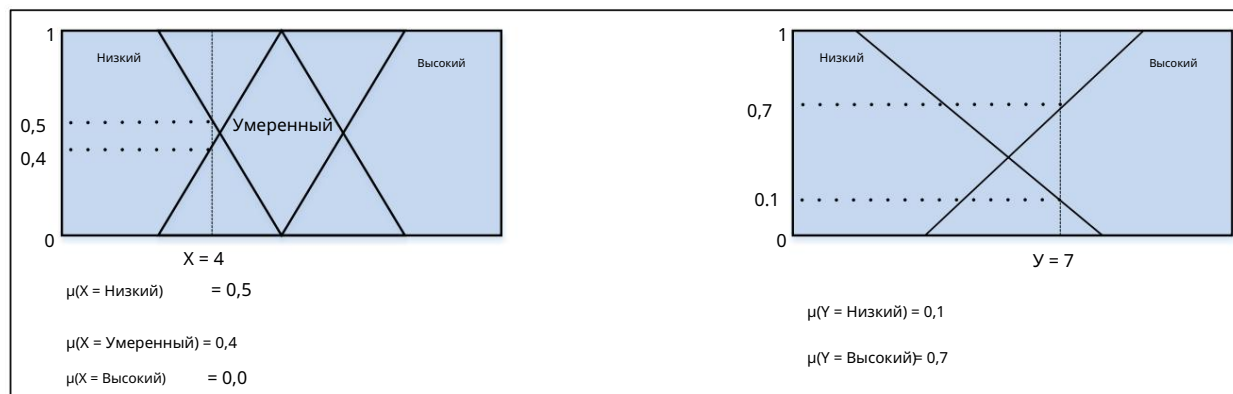


Рисунок 5. Фазификация

Шаг 2: Оценка правил

На этом этапе все три правила оцениваются для каждого нечеткого набора Сложности, Неопределенности и Усилий по Разработке, как показано на рисунке 6.

Условные предложения в наборе, основанном на правилах, были математически вычислены

следующим образом: « X OR Y =

$\text{MAX}(X, Y)$ » и « X AND Y = $\text{MIN}(X, Y)$ »

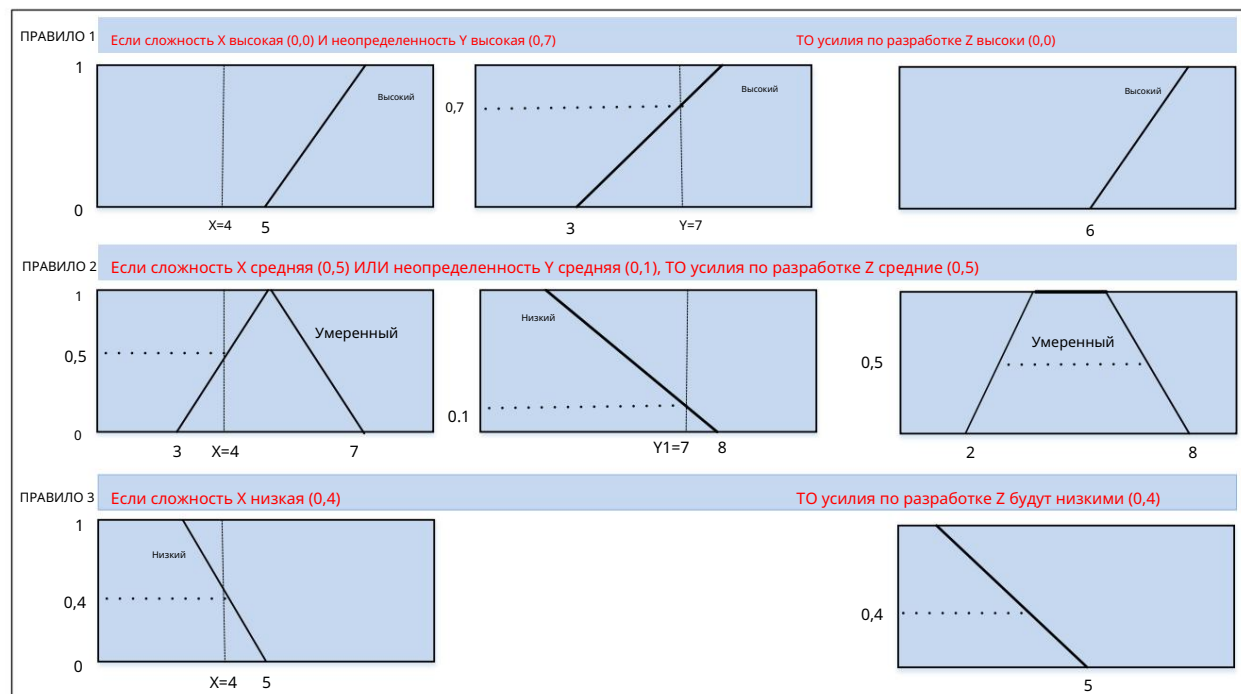


Рисунок 6. Оценка правил

Шаг 3: Оценка результата выполнения правила

На этом этапе все три выходных значения агрегируются. Как показано на рисунке 7, агрегация — это процесс объединения функций принадлежности всех правил (низкого, среднего и высокого уровня), ранее отсеченных или масштабированных, в единый нечёткий набор.

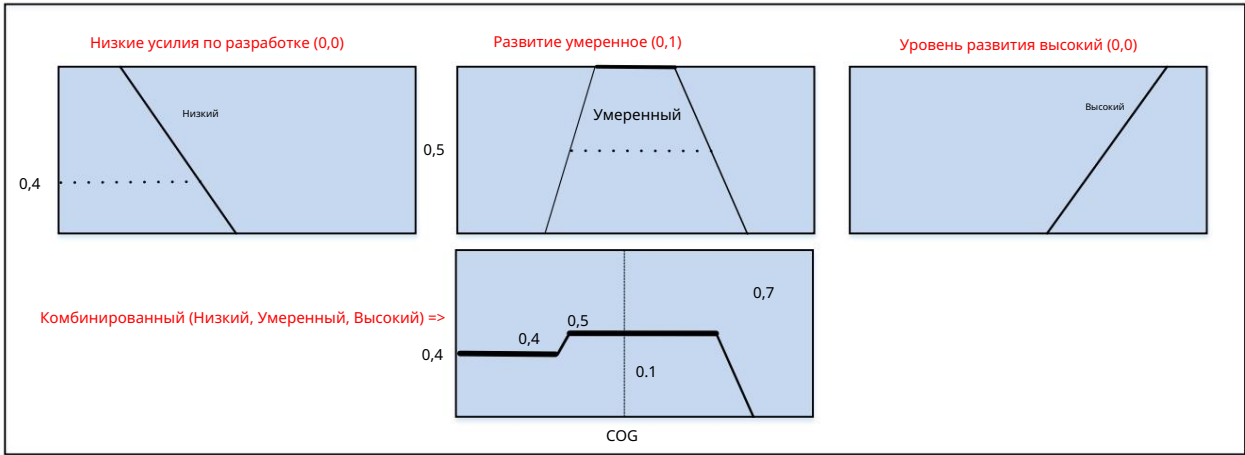


Рисунок 7. Оценка выходных данных

Шаг 4: Дефазификация

Для оценки базы правил использовалась нечёткость. Чтобы быть полезным в системе моделирования, конечный результат должен быть преобразован обратно в чёткий вывод. Этот процесс известен как дефазификация. Существует несколько методов дефазификации, таких как центральные суммы, среднее значение максимумов и лево-правый максимум, но наиболее популярным является центроид. Этот метод определяет точку, в которой вертикальная линия разделит совокупность на две равные массы. Математически эта точка представляет собой центр тяжести (ЦТ) и может быть выражена следующим образом:

= _____

Как видно из приведенного выше примера, метод Мамдани FLC представляет собой простой метод оценки правил нечеткой логики. Несмотря на простоту этого метода, он остается эффективным, поскольку использует набор правил, основанных на лингвистических терминах, которые эквивалентны человеческому мышлению и принятию решений.

IV. Вариант использования

Чтобы облегчить обсуждение, мы проиллюстрируем концепцию на примере использования: Agile-команда, ответственная за реализацию работ, связанных с моделированием и имитацией датчиков самолета.

Таблица 2 представляет собой подмножество фактических историй, накопленных командой разработчиков программного обеспечения, ответственной за разработку и внедрение программного обеспечения для моделирования датчиков воздушного судна. Эти истории отражают реальную работу по разработке, которую команде необходимо было оценить. Каждая история в этом бэклоге имела разный уровень сложности. Объем работ, необходимых для завершения реализации каждой истории, варьировался от истории к истории. Наконец, неопределенность каждой истории представляла собой потенциальный риск и препятствие, которые команде необходимо было учитывать в ходе цикла разработки.

Таблица 1. Подмножество отставания — программное обеспечение для моделирования датчиков самолетов

Story backlog
1- Radar – Remove SAR/ISAR mode
2- Configure Pre-Solic radar to use Max-M-Eyes build
3- Radar – Add weather mode
4- Radar buffer saved to shared memory
5- Upload radar state PDU
6- Add an operator “Manual track while scan” command

V. Реализация . Для

каждой истории Agile-команда должна оценить объем необходимых усилий (в соответствии с пунктами истории) для ее завершения и поставки. Для оценки объема усилий каждый член команды должен учитывать три основных фактора: сложность, объем работы и неопределенность или риск. Хотя сама концепция проста, реализовать ее непросто.

Например, разработчик может легко определить сложность конкретной задачи или степень неопределенности. Однако при попытке оценить трудозатраты, учитывая все три фактора одновременно, задача оценки не всегда проста.

В этом примере использования мы продемонстрируем, что, хотя оценка трудозатрат должна учитывать все три основных фактора совместно, каждый фактор можно оценить отдельно, а затем использовать процесс нечёткого вывода для объединения всех этих факторов для получения окончательной оценки. Для этого гибкой команде необходимо выполнить следующие две основные задачи: 1. Определить нечёткие переменные. Нечёткие лингвистические переменные определяются следующим образом: а. Неопределённость – шкала Лайкерта от 0 до 20: Низкая: линейная функция с отрицательным наклоном [0–20] и Высокая: линейная функция с положительным наклоном [0–20].

б) Сложность – шкала Лайкерта от 0 до 20: низкая [0–8], средняя [6–14], высокая [12–20]

с. Количество работ – шкала Лайкерта от 0 до 20: низкое [0–8], среднее [6–14], высокое [12–20]

г. Уровень усилий в баллах по шкале Лайкерта от 0 до 20: очень низкий [0–2], низкий [0–8], средний [5–13], высокий [12–20] и очень высокий [18–20]

На рисунке 8 показано графическое представление определённых выше нечётких переменных. Линейная функция и треугольная форма используются в данном приложении для облегчения калибровки между двумя подходами к оценке (например, планировщиком покера и нечёткой логикой).

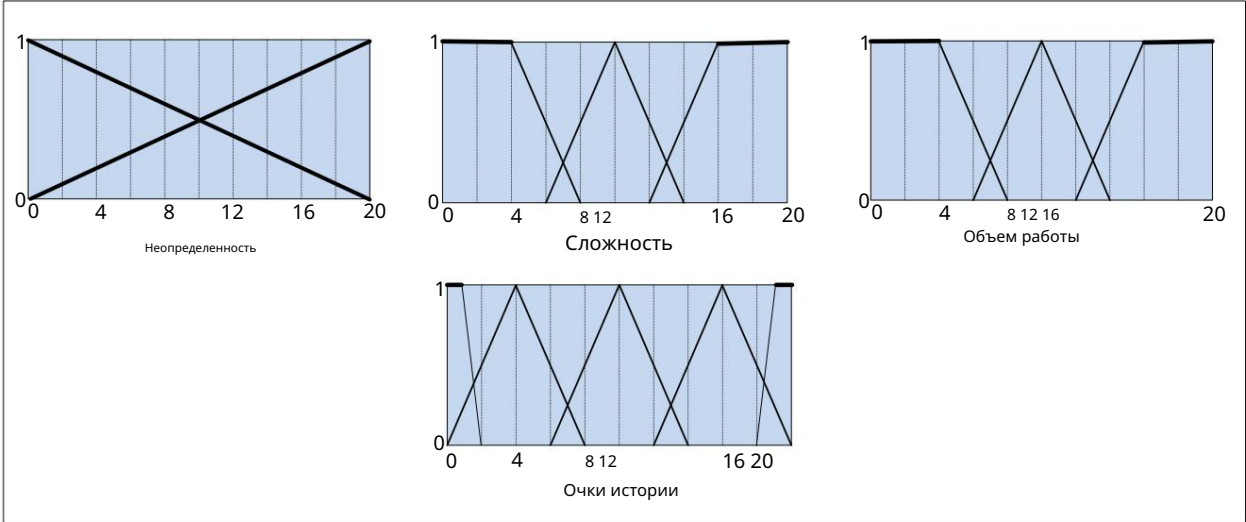


Рисунок 8. Определение нечетких переменных неопределенности (U), сложности (C), объема работы (W) и баллов истории

2. Построение базы данных на основе экспертных правил. Типичное экспертное правило: «если сложность очень низкая, неопределённость низкая и объём работы низкий, то уровень трудозатрат очень низкий». Было построено восемнадцать правил (2 x 3 x 3) с использованием всех возможных комбинаций входных нечётких переменных (например, неопределённости, сложности и объёма работы). Результирующие правила представлены в таблице 1. Тёмные ячейки представляют собой «истинное» условие. Например, правило № 1 должно выглядеть так: «если неопределённость низкая, сложность низкая и объём работы низкий, то количество баллов истории очень низкое».

Таблица 2. Нечеткие правила для усилий по разработке программного обеспечения (VL: очень низкий, L: низкий, M: средний, H: высокий, VH: очень высокий)

Rule #	Uncertainty		and	Complexity			and	Amount of Work			then	Story Points				
	L	H		L	M	H		L	M	H		VL	L	M	H	VH
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																
17																
18																

Нечеткое множество, показанное на рисунке 8, и правила в таблице 2 были реализованы с использованием инструмента системы нечеткого вывода (FIS), доступного в приложении MATLAB®.

Наконец, команде было предложено использовать модифицированную последовательность Фибоначчи для оценки каждой истории с помощью метода покера планирования. При использовании этого метода необходимо совместно учитывать неопределенность, сложность и объем работы для каждой истории. После завершения оценки с помощью покера планирования команде было предложено последовательно оценить неопределенность, сложность и объем работы для каждой истории, используя ту же модифицированную последовательность Фибоначчи. Благодаря этому новому подходу общая оценка трудозатрат в баллах может быть получена с помощью нечеткого множества и правил, определенных ранее.

На рисунке 9 показан пример оценки нечетких правил для истории № 2, описанной в таблице 2.

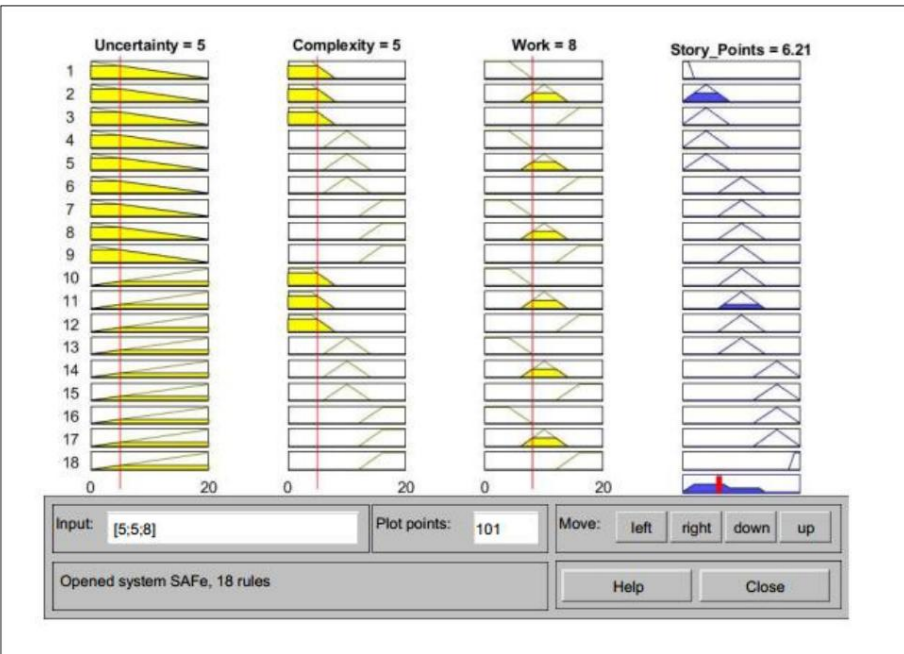


Рисунок 9. Оценка нечетких правил для истории №2 (U=5, C=5, W=8)

VI. Результаты

В таблице 3 сравниваются трудозатраты для обоих подходов. Столбец «Planning Poker» (SP0) представляет трудозатраты, оцененные в пунктах истории с использованием метода Planning Poker. Столбец «Fuzzy Logic» (SP1) представляет трудозатраты, оцененные в пунктах истории с использованием предлагаемого метода, основанного на нечеткой логике. Столбцы U, C и W представляют собой оценку неопределенности, сложности и объема работы соответственно для каждой истории.

Таблица 3. Оценка трудозатрат на разработку в Story Point для методов Poker Planner и Fuzzy Logic

Story	Poker Planner	Fuzzy Logic			
	SP0	SP1	U	C	W
1- Radar – Remove SAR/ISAR mode	3	3	1	1	5
2- Configure Pre-Solic radar to use Max-M-Eyes build	8	8	5	5	8
3- Radar – Add weather mode	5	5	1	8	5
4- Radar buffer saved to shared memory	8	8	3	5	8
5- Upload radar state PDU	3	3	1	3	3
6- Add an operator “Manual track while scan” command	20	15	5	15	20

Результат был рассчитан с использованием COG агрегации набора баллов истории. Для этой истории расчетная оценка трудозатрат составляет 6,21 балла истории. Для соответствия методу Planning Poker результат округляется до следующего целого значения модифицированной последовательности Фибоначчи, то есть до 8.

Из полученных результатов можно сделать вывод, что оценки, полученные с помощью обоих методов, очень хорошо совпадают, за исключением истории № 6 в Таблице 3. Расхождение можно объяснить тем, что команда могла переоценить ситуацию, используя метод планировщика Покера для оценки этой истории, поскольку этот метод требует от команды одновременного учета трех параметров (например, неопределенности, сложности и объема работы).

VII. Заключение

В данной статье представлен новый подход к оценке трудозатрат при разработке программного обеспечения. Поскольку мы столкнулись с задачей оценки, требующей приблизительного, а не точного, базового способа рассуждения, введение нечеткой логики в оценку трудозатрат было необходимо. Кроме того, предлагаемый подход использовал лингвистические переменные, которые гораздо лучше коррелируют с обычным человеческим способом рассуждения.

Члены команды разработчиков считали, что им проще оценить трудозатраты, последовательно рассматривая один параметр за раз, а не три параметра одновременно. Поэтому модель оценки трудозатрат на разработку программного обеспечения, включающая нечеткую логику, может помочь командам разработчиков преодолеть проблему одновременной оценки нескольких параметров (например, сложности, неопределенности и объема работы), когда определение каждого параметра, требующего человеческого суждения, расплывчато.

Ссылки: Синхал,

- [1]. А., Верма, Б. (2013). Разработка программного обеспечения: обзор. Международный журнал передовых исследований в области компьютерных наук и программной инженерии. Том 3 (6): 1120–1135.
- [2]. Авасти, Р., Батра, Б., Кундра, Х. (2016). Гибридная параметрическая модель для обогащения оценки трудозатрат на разработку программного обеспечения. Международный журнал компьютерных наук и информационной безопасности. Том 14 (10).
- [3]. Бём, Б. (1981). Экономика программной инженерии. Энглвуд Клиффс, Нью-Джерси: Prentice Hall.
- [4]. Чавла, Р., Ахлават, Д. (2015). Разработка программного обеспечения с использованием фреймворка нечеткой логики – Реализация. Международный журнал передовой компьютерной теории и инженерии. Том 4 (4).
- [5]. Камал, С., Камал, Э., Хан, С., Насил, Дж. А. (2013). Модель оценки программного обеспечения на основе нечеткой логики. Международный журнал по программной инженерии и её применению. Том 7, № 2.
- [6]. Су, М.Т., Линг, Т.С., Фан, К.К., Лю, Ч.Х., Ман, П.В. (2007). Повышение эффективности разработки программного обеспечения и оценка стоимости с использованием модели нечеткой логики. Малайзийский журнал компьютерных наук. Том 20(2).
- [7]. Леффингвелл, Д. (2018). Справочное руководство SAFe 4.5. Масштабируемая гибкая методология для бережливых предприятий. Второе издание. Издатель: Addison-Wesley Professional.
- [8]. Леффингвелл, Д., Якима, А., Кнастер, Р., Джемило, Д., Орен, И. (2017). Scaled Agile Framework для бережливого программного обеспечения и системной разработки. Scaled Agile, Inc.
- [9]. Георгссон, А. (2011). Внедрение Story Points и пользовательских историй для оценки в организациях, занимающихся разработкой программного обеспечения. Магистерская диссертация, Университет UMEÅ, факультет вычислительной техники.
- [10] Хамуда, А.Е. (2014). Использование точек Agile Story как метода оценки в организациях, использующих CMMI. Конференция Agile 2014.
- [11]. Магдалена, Л. (2015). Нечеткие системы с линейчатыми правилами. Springer Handbook of Computational Intelligence. Глава 13. С. 203–218.
- [12] Заде ЛА (1965). Нечеткие множества. Международный журнал информации и управления. Том 8. Выпуск 3: 338–353.
- [13] Мамдани, Э. Х., Ассилиан, С. (1975). Эксперимент по лингвистическому синтезу с нечетким логическим контроллером, Int. J. Man-machine Studies 7: 1–13.
- [14] Сугено, М., Канг, Г.Т. (1988). Структурная идентификация нечетких моделей. Нечеткие множества и системы 28: 15–33.
- [15]. Цукамото, Й. (1979). Подход к методу нечетких рассуждений. В книге под редакцией Мадана М. Гупты, Раммохана К. Рагаде и Рональда Р. Ягера «Достижения в теории нечетких множеств и её применении», Северная Голландия, Амстердам: 1979, стр. 137–149.
- [16]. Клир, Г. Дж., Юань, Б. (1995). Нечеткие множества и нечеткая логика. Prentice Hall PTR.